# Testing Phase: Building a Blog with Vite + React, Strapi, and Tailwind CSS

Tyler Sanderson

Wilmington University

Experiential Learning in CSC (22672.SEM)

Charles A. Bachman, PhD MBA PMP

April 16, 2025

Abstract

The blog platform was developed to serve as a dynamic documentation tool for personal computer science projects and academic growth. Built using Vite with React, Tailwind CSS for styling, and Strapi for backend functionality, it aimed to showcase skills and knowledge while ensuring accessibility across various devices. Development faced challenges such as route management via Slug instead of documentId, integrating social media features like React-Share, enhancing search functionality through improved GraphQL queries with Meilisearch, and addressing performance issues during testing. Testing phases included integration using Apollo hooks, E2E testing for user actions like login and publishing, and UAT across different devices. Issues encountered during testing, such as content wrapping adjustments, code block rendering, and search functionality, were partially addressed through further research into TailwindCSS plugins and Meilisearch. Despite these challenges, the platform successfully validated most business requirements, demonstrating progress in responsive design and core functionalities.

**Testing Phase: Building a Blog with Vite + React, Strapi, and Tailwind CSS**

Utilizing a modern tech stack - Vite+React, TailwindCSS, and Strapi, this project aims to develop a blog platform that can be easily maintained over time and serve as extra documentation to users' projects and portfolios as they learn and continue to grow; additionally, serving as a dynamic and interactive showcase of the skills and expertise I've learned throughout my computer science program. The future target audience for whom may be developers, students, or professionals looking for an interactive and visually appealing way to document their work and learning, but will be served with the smaller ambitions of 2 home users for the time being, until later sprints are iterated to for a larger VPS deployment.

Due to a few setbacks in our knowledge and understanding of our stack, discussed further in our Development phase, the beginning of the testing phase truly marked the beginning of our front-end development, and circumstantially provided many more opportunities for learning and testing!

To keep this document brief, we will skip a lot of the minute-technical challenges we faced during development, as a lot can be gleaned from our now public GitHub! We will instead discuss the methodologies used to conduct testing during the development of our front-end and the challenges faced in order to validate and further research the functionality of our blog platform against our requirements; specifically, ensuring the platforms robustness, usability, and reliability across various devices and user interactions throughout our application.

To conduct our testing, we spent some time analyzing our business requirements, as well as conducting end-to-end testing, integration testing, and user-acceptance testing.
As documented in our analysis phase, the requirements were as follows:

| Functional Requirements | | Non-Functional Requirements | |
|---|---|---|---|
| **Blog** | | **Capacity** | |
| The website shall display an About-Me Section. | Must Have | The website shall support a peak of 50 or less concurrent sessions at launch. | MustHave |
| The website shall display a Blog Section with recent posts. | Must Have | **Availability** | |
| The website shall display Social Media Sharing buttons. | Must Have | The system should be available 24 hrs per day, 334 days a year. | MustHave |
| The website shall display Portfolio/Project Links. | Must Have | The system shall not lose any user content. | MustHave |
| The website shall be sortable by Categories/Tags. | MustHave | The system shall reflect user changes within 5 minutes of | MustHave |
| | | submission. | |
| The website must allow Embedded Code. | MustHave | The website should log users in within 5 seconds. | MustHave |
| The website shall have a Search Function. | MustHave | | |
| The website should have a Comments Section per blog post. | ShouldHave | | |
| The website should feature Responsive Design to accommodate a variety of devices. MustHave. | MustHave | | |
| The website must feature SEO Optimization | MustHave | | |

In our Development phase, we found the About-Me to be redundant for current iterations, given the short bio on our homepage. This iteration found the portfolio/project links pages to be a bit redundant given the initial scope and links to our qualifying GitHub and LinkedIn in our

created Navbar. Some user deliberation found that even at current scope, the About-Me and portfolio buttons/links could be consolidated into a more comprehensive separate page to be more visually appealing, informative, while maintaining the removal of redundancy. This page could store a more informative About-Me, as well as a formatted copy of our working resume, and some more specific cards to feature highlighted projects and interest! This idea seems more fitting than being separated components and sections, as well as it reduces the user's need to click around, which will be especially important for accessibility and screen-readers.

Initially, we intended the social media sharing buttons to link and pre-fill visitors posts on their respective platforms, enabling better discovery once our app is deployed. While this is still something future iterations hope to solve, more research is needed on React-Share as an npm project dependency. The TL:DR being that we should be able to utilize the React-Share library in order to import all the required iconography, as well as allow us to quickly pass props such as our URL and title to our share button components so that users can quickly share our posts as the website gains more traction (Larocca, 2024). As the website is still currently being worked on to improve functionality, namely routing via Slug instead of documentId for readability and memorability, this feature has been pushed for the time being until said underlying functionalities can be better implemented in our URLs.

While Strapi only seems to be able to directly pass on a documentId from our useParams field on our article page, it doesn't seem to be able to call the slug as a modifier in our GraphQL request without using a filter. Strapi's documentation mentions filters in its REST API documentation (Strapi, REST API: Filters), but it seems a bit lacking for current versions of Strapi. Some more research revealed our call may look similar to "http://localhost:1337/api/posts?filters[slug][$eq]='slug'" (Myddna, 2021); otherwise we'll have

to modify Strapi Query Engine's 'routes' and 'controller' functions, which will require much more research it seems depending on the implementation (Hamza, 2024) (Ovinz et al., 2023).

Similarly, the search function has been commented out until we can gain a better understanding of how Strapi processes queries via GraphQL, as well as all the possible 'catch' blocks we may need to implement in order to address any issues that might arise when allowing unpredictable user input. Normally, I would just add some transformations and requirements for input; but I want users to be able to input whatever they like into our search bar, querying broadly across our posts. Upon failure to find any meaningful relevancy from the users input, I still want the application to serve some of the websites recent articles to keep the user engaged. This seems to still be possible using Strapi's Query Engine and filters, but maybe limits us to just a few filters for search. To implement better typo-tolerance, search filtering, semantic search, and predictive search, we may want to implement a plugin like Meilisearch for our final implementation once we're able to route via slug. As said, more research is still needed for proper implementation, but Meilisearch have even provided some documentation regarding integrating with Strapi and they've included an importable tool called "instant-meilisearch" for our front-end to help render our search bar and filtering (Vermandel, 2021)! This approach will better ensure proper usability and resiliency in our search bar, so this seems to be the most optimal approach in regards to functionality and time-savings.

Since the beginning of this phase, essentially the beginning of our front-end development, we've stumbled across many different challenges and errors, many of which have now been briefly detailed in our development branch commits to our public GitHub.

**GitHub:** https://github.com/sweettigertea/VRTS-dev-blog

These setbacks have provided many exemplary testing and exciting learning

opportunities throughout! In this time, 20 days or so, we've learned to completely render our two main pages detailed in our Design phase while overcoming and learning to understand many of the errors and implementations we've come across! We stumbled across many console errors related to Vite's strict out-of-the-box controls, which we've opted to keep for learning purposes. We learned to use react-router-dom with Strapi API calls to render our pages. We learned more about TailwindCSS and it's various markdown capabilities and plugins (Smoljames, 2024). We learned to better manage our code base using Prettier's Tailwind plugin and how to apply it to our existing code (Reinink & Wathan, 2022). We ran into some left over dependency errors from Husky and Git when installing our BlocksRenderer, which we've learned to read and understand to clear up current and future conflicts. We learned to use BlocksRenderer to dynamically display our content from our rich text editor, which we can now further customize with Tailwind's markdown plugin to display dynamic content how we intend, and so much more (Strapi, Strapi Blocks React Renderer)!

In short, while we have the confidence and ability to add proper category sorting using GraphQL API calls and routing now, the time constraints as of now have deemed this implementation more appropriate for an added iteration. Additionally, with the lack of current articles on the site, sorting seems a bit unnecessary for usability, but is still immediately important to properly demonstrating the skills and expertise I've learned throughout my computer science program, so this should come first in our features backlog.

To implement category sorting, we can actually utilize the same methodology used to render our articles through our react-router-dom and using GraphQL relational API calls (Ninja, 2021). This provides a number of benefits. We can add better SEO for our categories as the website grows by utilizing a separate page with separate URLs for each category, making this

feature more memorable and searchable; and this method will make optimizations for screen-readers more clear while still being dynamic. Since screen-readers are essentially lists, this will create more surface level lists, instead of having to navigate back-and-forth through one big lists with dynamic sub-lists, reducing the amount of clicking and waiting around needed to interact with specific features. However, adjustments of our schedule may still allow this feature to be implemented before the projects deadline on May 4th!

Lastly, one of our requirements was SEO optimization, as this is websites purpose is to dynamically showcase ones portfolio and interest while networking with others. Again though, due to further research and development required to implement proper slug routing, as well as the security concerns that may come with deploying a project publicly, this requirement has also been pushed until after the Implementation and Deployment phase to reduce web traffic until security concerns can be properly assessed and managed. Additionally, after further research, Search Engine Optimization seems to a much larger topic than anticipated and can probably be reserved to own iteration by itself after some of the other requirements have been met.

With all that said, we were able to implement many of the features we laid out in our business requirements during our Analysis phase. We utilized Integration testing, End-to-End (E2E) testing, and finally User-Acceptance testing to validate our the remaining business requirements; the current scope of which includes our blog section with recent articles, responsive design, embedded code and image support, and user application performance.

Our integration testing was performed throughout the development process by using Apollo hooks to make GraphQL queries. We used console logs to manage where our calls were coming from and to ensure efficient data fetching was coming with our responses when rendering our content.

```
console.log("Homepage");
// console.log(ArticlesTest)
// console.log('gql')
console.log(data.articles);
// const articles=[
//     {
//         'id':1,
//         'title':'Article Title',
//         'desc':'Lorem ipsum dolor sit amet consectetur adipisicing elit. Suscipit, fugit fuga ab beat
//         'coverImg':imgPlaceholder,
//         'contents':'Lorem ipsum dolor sit amet consectetur adipisicing elit. Nulla distinctio libero
//         'authorName':"Author Name",
//         'authorImg':imgPlaceholder,
//         'authorDesc':'Author Description',
//     },
```

```
Homepage
                                                    Homepage.jsx:57:11
▼Array(7) [ {…}, {…}, {…}, {…}, {…}, {…}, {…} ]
  ▶0: Object { __typename: "Article", documentId:
  "bi8poiahxvzldxeqa2t1ab1n", slug: "beautiful-picture", …
  }
  ▶1: Object { __typename: "Article", documentId:
  "cgmh47io68quli7u7pezts2k", slug: "the-internet-s-own-
  boy", … }
  ▶2: Object { __typename: "Article", documentId:
  "co2k7w76eeu6iw1sst7teqm4", slug: "this-shrimp-is-
  awesome", … }
  ▶3: Object { __typename: "Article", documentId:
  "pe1nn8zhsly2krwp9j8dsn48", slug: "what-s-inside-a-black-
  hole", … }
  ▶4: Object { __typename: "Article", documentId:
  "hzr5zmg9sz5spplh5u7veyi9", slug: "a-bug-is-becoming-a-
  meme-on-the-internet", … }
  ▶5: Object { __typename: "Article", documentId:
  "gukq68l9subcd7i15eum7jt3", slug: "bingbong", … }
  ▶6: Object { __typename: "Article", documentId:
  "d1k94skrrkqla1y4d1355nf9", slug: "test", … }
    length: 7
```

```
Article Details
                                                    Article.jsx:62:11
▼Object { __typename: "Article", documentId:
"hzr5zmg9sz5spplh5u7veyi9", slug: "a-bug-is-becoming-a-
meme-on-the-internet", title: "A bug is becoming a meme on
the internet", description: "How a bug on MySQL is becoming
a meme on the internet", cover: {…}, author: {…}, content:
(6) […] }
    __typename: "Article"
  ▶author: Object { __typename: "Author", name: "Sarah
  Baker", email: "sarahbaker@strapi.io", … }
  ▶content: Array(6) [ {…}, {…}, {…}, … ]
  ▶cover: Object { __typename: "UploadFile", url: "/
  uploads/
  a_bug_is_becoming_a_meme_on_the_internet_6e8d10fb59.jpeg"
  }
    description: "How a bug on MySQL is becoming a meme on
    the internet"
    documentId: "hzr5zmg9sz5spplh5u7veyi9"
    slug: "a-bug-is-becoming-a-meme-on-the-internet"
    title: "A bug is becoming a meme on the internet"
```

Using GraphQL, we ensure we're getting only the data we ask for, which ensures more efficient data retrieval! We also used Firefox's built-in developer tools to test our applications load times under different throttling conditions. Under DSL conditions, which is still be pretty common around the world for loading web apps, the website took about 10 whole seconds to load with no caching enabled. Coincidentally, under the same conditions, GitHub takes about the same amount of time, but they load some of the assets ahead of time to make the experience feel less frustrating. This indicates that while our load time for the full application may actually be decently performant, there's more research to do in regards to making the site load more efficiently and make it more engaging while loading. Perhaps we should utilize more static

placeholders with loading animations before loading all our request. This may make the experience more bearable for users with slower internet connections, but may also add a bit of processing and call-time, so further research will need to be done concerning the methodologies of static loading in our stack, TailwindCSS animated icons and their benefits or drawbacks in regards to less performant clients, and the structuring of skeleton framing for our site to load more gradually. We could also add a spinner to our placeholder 'Loading…' to call the users attention when we're making our request to make it more apparent that is what we're waiting for.

Additionally, tying into our E2E testing, we utilized placeholder JSON test as well as various other integration test as we learned more about the various components of our chosen stack; many of which will be left commented in our development branch for preservation and continuous learning.

We made use of End-to-End (E2E) testing throughout the development of our application to ensure features behaved as expected as we implemented them. While other unexpected errors may occur during the projects lifetime, by simulating our users experience we can verify key components aren't being overstepped in regards to time-constraints and we can hone in when bugs occur, ruling out some potentials as we experiment and test. For the time being, we tested the current complete application flow from back-end to front-end. We created another Author, we created another entry, and tested leaving various values blank. This research led us to learn more about Strapi's 'required' fields in order to prevent null values and we've started further research into creating more engaging catch values for when important data calls may fail to respond.

We also tested various components of our BlocksRenderer in regards to embedded images and code blocks, in which we spent some more time researching markdown and how our

TailwindCSS-Typography plugin cycles through our rich text components in order to properly render our images and code blocks, more on the plugin later.

Lorem ipsum dolor sit amet consectetur adipiscing elit. Quisque faucibus ex sapien vitae pellentesque sem placerat. In id cursus mi pretium tellus duis convallis. Tempus leo eu aenean sed diam urna tempor. Pulvinar vivamus fringilla lacus nec metus bibendum egestas. Iaculis massa nisl malesuada lacinia integer nunc posuere. Ut hendrerit semper vel class aptent taciti sociosqu. Ad litora torquent per conubia nostra inceptos himenaeos.

```
<div className="prose max-w-3xl pt-6 wrap-anywhere indent-8">
<BlocksRenderer let content={article.content} />
</div>
```

Lorem ipsum dolor sit amet consectetur adipiscing elit. Quisque faucibus ex sapien vitae pellentesque sem placerat. In id cursus mi pretium tellus duis convallis. Tempus leo eu aenean sed diam urna tempor. Pulvinar vivamus fringilla lacus nec metus bibendum egestas. Iaculis massa nisl malesuada lacinia integer nunc posuere. Ut hendrerit semper vel class aptent taciti sociosqu. Ad litora torquent per conubia nostra inceptos himenaeos.

Lorem ipsum dolor sit amet consectetur adipiscing elit. Quisque faucibus ex sapien vitae pellentesque sem placerat. In id cursus mi pretium tellus duis convallis. Tempus leo eu aenean sed diam urna tempor. Pulvinar vivamus fringilla lacus nec metus bibendum egestas. Iaculis massa nisl malesuada lacinia integer nunc posuere. Ut hendrerit semper vel class aptent taciti sociosqu. Ad litora torquent per conubia nostra inceptos himenaeos.

We then spent some time navigating and testing various view ports and actions to ensure our changes scaled well during our development.

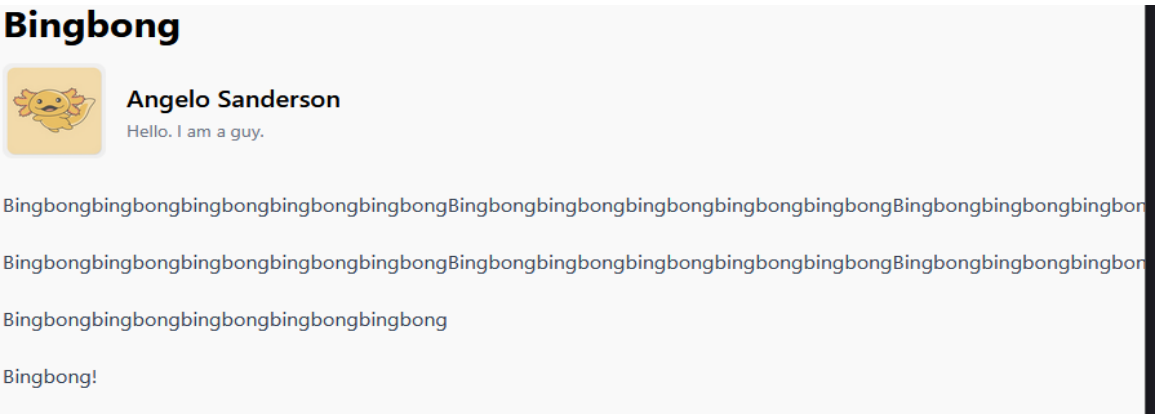
Finally, we utilized User Acceptance Testing, or UAT, to validate our own E2E testing in regards to responsive design, Login time, and publishing time. This test case involved logging into Strapi, navigating through content management to create an entry and author, inputting details, publishing the article, and checking functionalities such as responsive design, published posts displaying, standard prose rendering, and navigation via navbar.

**sttDevBlog – End-2-End (E2E) User Acceptance Testing (UAT)**

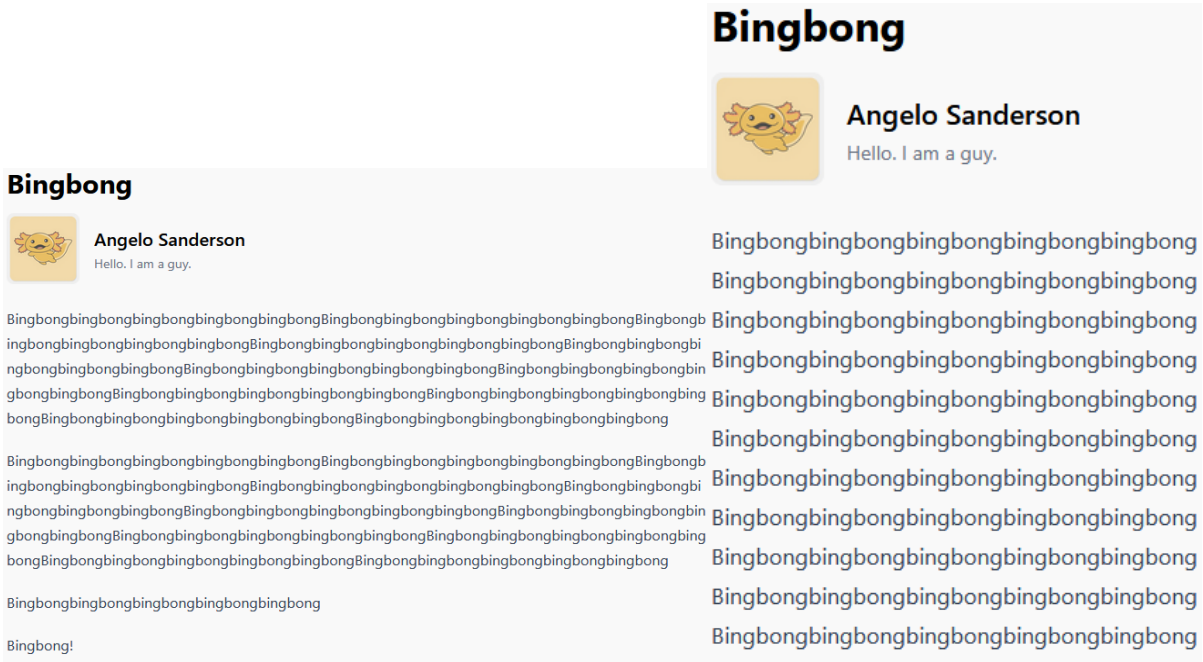
| | |
|---|---|
| **Release #** | Week 11 – Testing Phase |
| **Process** | Writing a blog post – Strapi Backend |
| **Test User** | SweetTigerTea's DevBlog: Tyler Sanderson |
| **Login Information** | Link: http://localhost:1337<br>Company: sttDevBlog<br>Username: xxx<br>Password: xxx |
| **Tester** | Angelo Sanderson |
| **Scenario Name:** | StrDBP - 4.6.25 |
| **Test:** | User creates blog post. Test user verifies post from front-end! |
| **Script Description:** | Our user will login into our Strapi portal and write a test blog post; then we will evaluate if said post shows up in our front-end per our business requirements. |

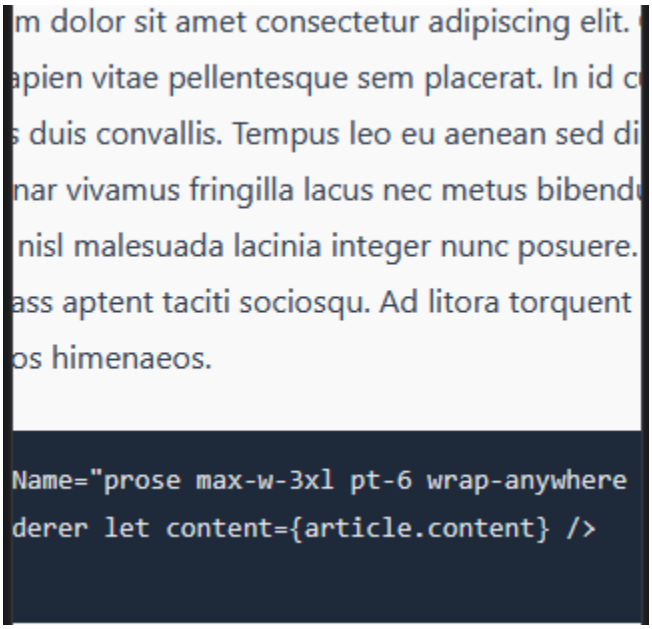
| Step # | Step | Data | Expected Outcome | Actual Results | Pass/Fail | Comments/Issues |
|---|---|---|---|---|---|---|
| 1 | Login to Strapi | **Login Info** | Logs user into Strapi Back-end | Logged into Strapi Back-end | Pass | |
| 2 | Navigate to content-management > Author > Create new entry! | | Allows user to create new entry! | User created a new entry | Pass | |
| 3 | Input name, description, email, and avatar. | **Avatar: Axolotl, or from upload...** | Creates new author! | User created new author. | Pass | |
| 4 | Navigate to Article > Create new entry! | | Allows user to create new article! | User was able to access create new entry | Pass | |
| 5 | Input title, slug (generated), description, author, cover, categories, and content! | | Allows user to create new Article. Generates slug automatically. | User was able to input article information, save, and publish. | Pass | Requires checkbox selection to select cover image. Need permissions for 'categories'... |
| 6 | Select Save, then Publish! | | Publishes post to the front-end! | User successfully published post to the front-end | Pass | |
| 7 | Navigate to "http://localhost:5174" | | Displays new article in card format on homepage. | New article was displayed in card format on homepage | Pass | |
| 8 | Click on newly created article to view your magnificent work! :o | | Displays article in standard prose! | Article text was displayed, but continuous text string did not wrap. | Fail | Text needs to wrap in case of continuous string of text |
| 9 | Click links in navbar. | | Navigates browser to Home, Github, or Linkedin respectively. | Navbar links navigated appropriately. | Pass | |
| 10 | Do whatever you want! Try to break it! | | Doesn't break! | Nothing broke! | Pass | |

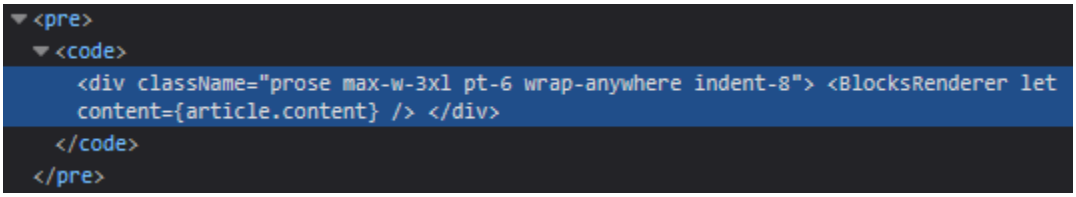
This test case actually revealed some interesting challenges! While our test passed in regards to responsive design in most cases, our tester actually found an egregious formatting error that forced our content outside the range of its container off of our view-port!



To fix this within our prose, we've added 'wrap-anywhere' to various elements across our app to account for continuous string of characters. TailwindCSS handles text wrapping by wrapping normal by default. This means that our content will dynamically adjust in order to render whole words until the view-port is too tiny, in which the text will break anywhere to wrap into the view-port, fixing our current issues with wrapping standard text within our prose!


Bingbong
Angelo Sanderson
Hello. I am a guy.
Bingbong
Angelo Sanderson
Hello. I am a guy.
Bingbongbingbongbingbongb




Additionally, there are a few elements such as the Index (Adegbemile, 2024) (Strapi Table of Contents Guide 2025), word count, and read time (Paidly) for our articles that have been commented out as we continue to research and experiment with different methodologies of auto-rendering said elements. None of the implementations researched seem inherently difficult to implement, so these issues may be cleared up within this iterations development; however, since these elements aren't necessary to the overall fit and function of the current application, so they've been commented out for the time being until proper implementation has been reached.

In regards to the current overall user experience and functionality of our stack, the website was able to login, publish, and display our content within seconds of update, invalidating our current concerns in relation to performance and usability from our user's perspective!

In conclusion, many more challenges presented themselves as learning and testing opportunities. The testing phase successfully validated most of our required functionalities, with

minor issues requiring attention to enhance the overall user experience, system robustness, and showcase of our ability and excitement to learn more! We discussed our use of Integration testing, End-2-End testing, and User Acceptance Testing to measure our successes against our business requirements and discussed some of the challenges and measures that have been taken, or will be taken, in order to address these issues in regards to usability and reliability across various devices and user interactions. In our Implementation and Maintenance phase, we will continue to implement features and discuss the progress and research we’ve made in securing and hardening our application for public release on a Virtual Private Server!

References

Adegbemile, D. (2024, June 3). ow to create a Table Of Contents in React and Strapi rich text editor. Medium. https://medium.com/@adegbemile/how-to-create-a-table-of-contents-in-react-and-strapi-rich-text-editor-1a6ff7b4d10e

Hamza, M. (2024, November 23). How to search by slug instead of ID in Strapi V4. Medium. https://medium.com/@malikhamzav/how-to-search-by-slug-instead-of-id-in-strapi-v4-2a66e24f0dab

Larocca, M. J. (2024, July 29). Implementing Dynamic Social Media Sharing in Your React App with React Share. SelfTaughtTXG. https://selftaughttxg.com/2024/07-24/implementing-dynamic-social-media-sharing-in-your-react-app-with-react-share/

Myddna. (2021, December). Strapi V4 search by slug instead ID. Strapi Community Forum. https://forum.strapi.io/t/strapi-v4-search-by-slug-instead-id/13469/6

Ninja, N. (2021, July 28). Strapi Crash Course (with React & GraphQL) #13 - Fetching Related Data. YouTube. https://www.youtube.com/watch?v=Ym-UBtxril4&list=PL4cUxeGkcC9h6OY8_8Oq6JerWqsKdAPxn&index=14

Ovinz, Oikon, A., & Hagel, T. (2023, April 3). How to Fetch Strapi by slug, and how to populate the categories. Stack Overflow. https://stackoverflow.com/questions/73261870/how-to-fetch-strapi-by-slug-and-how-to-populate-the-categories

Paidly. (n.d.). Strapi Plugin Reading Time. GitHub. https://github.com/Paidly/strapi-plugin-reading-time

Reinink, J., & Wathan, A. (2022, January 24). Automatic Class Sorting with Prettier. TailwindCSS. https://tailwindcss.com/blog/automatic-class-sorting-with-prettier

Smoljames. (2024, March 14). Learn TailwindCSS in 23 minutes. YouTube. https://www.youtube.com/watch?v=W-LDhPyv478

Strapi Table of Contents Guide. Restackio. (2025, April 18). https://www.restack.io/p/strapi-knowledge-strapi-table-of-contents-guide

Strapi. (n.d.-a). REST API: Filters. Strapi 5 Documentation. https://docs.strapi.io/cms/api/rest/filters

Strapi. (n.d.-b). Strapi Blocks React Renderer. GitHub. https://github.com/strapi/blocks-react-renderer

Tailwindlabs. (n.d.). Tailwind CSS Typography. GitHub. https://github.com/tailwindlabs/tailwindcss-typography

Vermandel, C. (2021, May 4). How to add relevant search to your Strapi app using Meilisearch. Meilisearch. https://www.meilisearch.com/blog/add-meilisearch-to-your-strapi-app